

Core Language Working Group "ready" Issues for the June, 2023 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4944](#).

170. Pointer-to-member conversions

Section: 7.3.13 [[conv.mem](#)] **Status:** ready **Submitter:** Mike Stump **Date:** 16 Sep 1999

The descriptions of explicit (7.6.1.9 [[expr.static.cast](#)] paragraph 9) and implicit (7.3.13 [[conv.mem](#)] paragraph 2) pointer-to-member conversions differ in two significant ways:

1. In a `static_cast`, a conversion in which the class in the target pointer-to-member type is a base of the class in which the member is declared is permitted and required to work correctly, as long as the resulting pointer-to-member is eventually dereferenced with an object whose dynamic type contains the member. That is, the class of the target pointer-to-member type is not required to contain the member referred to by the value being converted. The specification of implicit pointer-to-member conversion is silent on this question.

(This situation cannot arise in an implicit pointer-to-member conversion where the source value is something like `&x: : f`, since you can only implicitly convert from pointer-to-base-member to pointer-to-derived-member. However, if the source value is the result of an explicit "up-cast," the target type of the conversion might still not contain the member referred to by the source value.)

2. The target type in a `static_cast` is allowed to be more cv-qualified than the source type; in an implicit conversion, however, the cv-qualifications of the two types are required to be identical.

The first difference seems like an oversight. It is not clear whether the latter difference is intentional or not.

(See also issue 794.)

CWG 2022-11-09

The second concern is NAD; implicit conversions allow chaining a pointer-to-member conversion with a qualification conversion.

Proposed resolution (approved by CWG 2023-02-06):

1. Change in 7.3.13 [[conv.mem](#)] paragraph 2 as follows:

A prvalue of type "pointer to member of B of type cv T", where B is a class type, can be converted to a prvalue of type "pointer to member of D of type cv T", where D is a complete

class derived (11.7 [class.derived]) from B. If B is an inaccessible (11.8 [class.access]), ambiguous (6.5.2 [class.member.lookup]), or virtual (11.7.2 [class.mi]) base class of D, or a base class of a virtual base class of D, a program that necessitates this conversion is ill-formed. **If class D does not contain the original member and is not a base class of the class containing the original member, the behavior is undefined. Otherwise, The the** result of the conversion refers to the same member as the pointer to member before the conversion took place, but it refers to the base class member as if it were a member of the derived class. The result refers to the member in D's instance of B. Since the result has type “pointer to member of D of type cv T”, indirection through it with a D object is valid. The result is the same as if indirecting through the pointer to member of B with the B subobject of D. The null member pointer value is converted to the null member pointer value of the destination type. [Footnote: ...]

2. Change in 7.6.1.9 [expr.static.cast] paragraph 13 as follows:

... If class B contains the original member, or is a base ~~or derived~~ class of the class containing the original member, the resulting pointer to member points to the original member. ...

1353. Array and variant members and deleted special member functions

Section: 11.4.5 [class.ctor] **Status:** ready **Submitter:** Sean Hunt **Date:** 2011-08-16

The specification of when a defaulted special member function is to be defined as deleted sometimes overlooks variant and array members.

Proposed resolution (approved by CWG 2023-02-07):

1. Change in 11.4.5.2 [class.default.ctor] paragraph 2 as follows:

A defaulted default constructor for class X is defined as deleted if:

- ~~X is a union that has a variant member with a non-trivial default constructor and no variant member of X has a default member initializer,~~
- ~~X is a non-union class that has a variant member M with a non-trivial default constructor and no variant member of the anonymous union containing M has a default member initializer,~~
- any non-static data member with no default member initializer (11.4 [class.mem]) is of reference type,
- any non-variant non-static data member of const-qualified type (or **possibly multi-dimensional** array thereof) with no *brace-or-equal-initializer* is not const-default-constructible (9.4 [dcl.init]),
- X is a union and all of its variant members are of const-qualified type (or **possibly multi-dimensional** array thereof),
- X is a non-union class and all members of any anonymous union member are of const-qualified type (or **possibly multi-dimensional** array thereof),
- any potentially constructed subobject, except for a non-static data member with a *brace-or-equal-initializer* **or a variant member of a union where another non-static data member has a brace-or-equal-initializer**, has class type M (or **possibly multi-dimensional** array thereof) and ~~either M has no default constructor or~~ overload resolution (12.2 [over.match]) as applied to find M's corresponding constructor **results in an ambiguity or in a function that is deleted or inaccessible from the defaulted default**

~~constructor~~ **either does not result in a usable candidate (12.2.1 [over.match.general]) or, in the case of a variant member, selects a non-trivial function,** or

- any potentially constructed subobject has ~~a type with~~ **class type M (or possibly multi-dimensional array thereof) and M has** a destructor that is deleted or inaccessible from the defaulted default constructor.

2. Change in 11.4.5.3 [class.copy.ctor] paragraph 10 as follows:

... A defaulted copy/move constructor for a class X is defined as deleted (9.5.3 [dcl.fct.def.delete]) if X has:

- a potentially constructed subobject of type M (or **possibly multi-dimensional** array thereof) ~~that cannot be copied/moved because for which~~ overload resolution (12.2 [over.match]), as applied to find M's corresponding constructor, ~~results in an ambiguity or a function that is deleted or inaccessible from the defaulted constructor~~ **either does not result in a usable candidate (12.2.1 [over.match.general]) or, in the case of a variant member, selects a non-trivial function,**
- ~~a variant member whose corresponding constructor as selected by overload resolution is non-trivial,~~
- any potentially constructed subobject of ~~a type with~~ **class type M (or possibly multi-dimensional array thereof) where M has** a destructor that is deleted or inaccessible from the defaulted constructor, or,
- for the copy constructor, a non-static data member of rvalue reference type.

3. Change in 11.4.6 [class.copy.assign] paragraph 7 as follows:

A defaulted copy/move assignment operator for class X is defined as deleted if X has:

- ~~a variant member with a non-trivial corresponding assignment operator and X is a union-like class, or~~
- a non-static data member of const non-class type (or **possibly multi-dimensional** array thereof), or
- a non-static data member of reference type, or
- a direct non-static data member of class type M (or **possibly multi-dimensional** array thereof) or a direct base class M that cannot be copied/moved because overload resolution (12.2 [over.match]), as applied to find M's corresponding assignment operator, ~~results in an ambiguity or a function that is deleted or inaccessible from the defaulted assignment operator~~ **either does not result in a usable candidate (12.2.1 [over.match.general]) or, in the case of a variant member, selects a non-trivial function.**

4. Change in 11.4.7 [class.dtor] paragraph 7 as follows:

A defaulted destructor for a class X is defined as deleted if:

- ~~X is a union-like class that has a variant member with a non-trivial destructor,~~
- any potentially constructed subobject has class type M (or **possibly multi-dimensional** array thereof) and M has a ~~deleted~~ **that is deleted** or ~~a destructor that is inaccessible from the defaulted destructor~~ **or, in the case of a variant member, is non-trivial,**
- or, for a virtual destructor, lookup of the non-array deallocation function results in an ambiguity or in a function that is deleted or inaccessible from the defaulted destructor.

Although the note in 7.2.1 [[basic.lval](#)] paragraph 1 states that

The discussion of each built-in operator in Clause 7 [[expr](#)] indicates the category of the value it yields and the value categories of the operands it expects

in fact, many of the operators that take prvalue operands do not make that requirement explicit. Possible approaches to address this failure could be a blanket statement that an operand whose value category is not stated is assumed to be a prvalue; adding prvalue requirements to each operand description for which it is missing; or changing the description of the usual arithmetic conversions to state that they imply the lvalue-to-rvalue conversion, which would cover the majority of the omissions.

(See also issue 1685, which deals with an inaccurately-specified value category.)

Proposed resolution (approved by CWG 2023-04-28):

1. Change in 7.2.1 [[basic.lval](#)] paragraph 6 as follows:

Whenever a glvalue appears as an operand of an operator that ~~expects~~ **requires** a prvalue for that operand, the lvalue-to-rvalue (7.3.2 [[conv.lval](#)]), array-to-pointer (7.3.3 [[conv.array](#)]), or function-to-pointer (7.3.4 [[conv.func](#)]) standard conversions are applied to convert the expression to a prvalue. ...

2. Change in 7.3.1 [[conv.general](#)] paragraph 1 as follows:

... A standard conversion sequence will be applied to an expression if necessary to convert it to **an expression having** a required destination type **and value category**. ...

3. Add to the bulleted list in 7.4 [[expr.arith.conv](#)] paragraph 1 as follows:

- ... This pattern is called the usual arithmetic conversions, which are defined as follows:
- The lvalue-to-rvalue conversion (7.3.2 [[conv.lval](#)]) is applied to each operand and the resulting prvalues are used in place of the original operands for the remainder of this section.
 - If either operand is of scoped enumeration type (9.7.1 [[dcl.enum](#)]), no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.
 - ...

4. Change in 7.6.1.3 [[expr.call](#)] paragraph 1 as follows:

... For a call to a non-member function or to a static member function, the postfix expression shall **be** either ~~be~~ an lvalue that refers to a function (in which case the function-to-pointer standard conversion (7.3.4 [[conv.func](#)]) is suppressed on the postfix expression), or ~~have~~ **a prvalue of** function pointer type.

5. Change in 7.6.2.2 [[expr.unary.op](#)] paragraph 7 as follows:

The operand of the unary + operator shall ~~have~~ **be a prvalue of** arithmetic, unscoped enumeration, or pointer type and the result is the value of the argument. Integral promotion is performed on integral or enumeration operands. ...

6. Change in 7.6.2.2 [[expr.unary.op](#)] paragraph 8 as follows:

The operand of the unary - operator shall ~~have~~ **be a prvalue of** arithmetic or unscoped enumeration type and the result is the negative of its operand. Integral promotion is performed on integral or enumeration operands. ...

7. Change in 7.6.2.2 [[expr.unary.op](#)] paragraph 10 as follows:

The operand of the ~ operator shall ~~have~~ **be a prvalue of** integral or unscoped enumeration type. Integral promotions are performed. ...

8. Change in 7.6.2.9 [[expr.delete](#)] paragraph 1 as follows:

... ~~The operand shall be of pointer to object type or of class type. If~~ **the operand is** of class type, ~~the operand~~ **it** is contextually implicitly converted (7.3 [[conv](#)]) to a pointer to object type. [Footnote: ...] **Otherwise, it shall be a prvalue of pointer to object type.** The *delete-expression* has type void.

9. Change in 7.6.4 [[expr.mptr.oper](#)] paragraph 2 and 3 as follows:

The binary operator .* binds its second operand, which shall be **a prvalue** of type “pointer to member of T” to its first operand, which shall be ...

The binary operator ->* binds its second operand, which shall be **a prvalue** of type “pointer to member of T” to its first operand, which shall be ...

10. Change in 7.6.6 [[expr.add](#)] paragraph 1 as follows:

The additive operators + and - group left-to-right. ~~The~~ **Each operand shall be a prvalue. If both operands have arithmetic or unscoped enumeration type, the** usual arithmetic conversions (7.4 [[expr.arith.conv](#)]) are performed ~~for operands of arithmetic or enumeration type.~~ **Otherwise, if one operand has arithmetic or unscoped enumeration type, integral promotion is applied (7.3.7 [[conv.prom](#)]) to that operand. A converted or promoted operand is used in place of the corresponding original operand for the remainder of this section.** ... For addition, either both operands shall have arithmetic ~~or unscoped enumeration~~ type, or one operand shall be a pointer to a completely-defined object type and the other shall have integral ~~or unscoped enumeration~~ type.

11. Change in 7.6.6 [[expr.add](#)] paragraph 2 as follows:

For subtraction, one of the following shall hold:

- both operands have arithmetic ~~or unscoped enumeration~~ type; or
- both operands are pointers to cv-qualified or cv-unqualified versions of the same completely-defined object type; or
- the left operand is a pointer to a completely-defined object type and the right operand has integral ~~or unscoped enumeration~~ type.

12. Change in 7.6.7 [[expr.shift](#)] paragraph 1 as follows:

... The operands shall be **prvalues** of integral or unscoped enumeration type and integral promotions are performed. ...

13. Change in 9.4.1 [[decl.init.general](#)] bullet 16.9 as follows:

- ...
- Otherwise, the initial value of the object being initialized is the (possibly converted) value of the initializer expression. A standard conversion sequence (7.3 [[conv](#)]) ~~will be~~ **is** used, ~~if necessary,~~ to convert the initializer expression to **a prvalue of** the cv-

unqualified version of the destination type; no user-defined conversions are considered.

- ...
- o ...

1973. Which *parameter-declaration-clause* in a *lambda-expression*?

Section: 7.5.5.2 [[expr.prim.lambda.closure](#)] **Status:** ready **Submitter:** Dinka Ranns **Date:** 2014-07-16

According to 7.5.5.2 [[expr.prim.lambda.closure](#)] paragraph 3,

The closure type for a *lambda-expression* has a public inline function call operator (for a non-generic lambda) or function call operator template (for a generic lambda) (12.4.4 [[over.call](#)]) whose parameters and return type are described by the *lambda-expression's parameter-declaration-clause* and *trailing-return-type* respectively, and whose *template-parameter-list* consists of the specified *template-parameter-list*, if any.

This is insufficiently precise because the *trailing-return-type* might itself contain a *parameter-declaration-clause*.

Suggested resolution [~~SUPERSEDED~~]:

Change in 7.5.5.1 [[expr.prim.lambda.general](#)] paragraph 5 as follows:

If a *lambda-declarator* does not ~~include a~~ **start with a parenthesized** *parameter-declaration-clause*, it is as if () were inserted at the start of the *lambda-declarator*. **A *lambda-expression's parameter-declaration-clause* is the (possibly empty) *parameter-declaration-clause* of the *lambda-expression's lambda-declarator*.** If the *lambda-declarator* does not include a *trailing-return-type*, the lambda return type is auto, which is deduced from return statements as described in 9.2.9.6 [[dcl.spec.auto](#)].

Proposed resolution (approved by CWG 2023-02-06):

1. Change in 7.5.5.2 [[expr.prim.lambda.closure](#)] paragraph 3 as follows:

The closure type for a *lambda-expression* has a public inline function call operator (for a non-generic lambda) or function call operator template (for a generic lambda) (12.4.4 [[over.call](#)]) whose parameters and return type are ~~described by~~ **those of** the *lambda-expression's parameter-declaration-clause* and *trailing-return-type* respectively, and whose *template-parameter-list* consists of the specified *template-parameter-list*, if any.

2. Change in 7.5.5.1 [[expr.prim.lambda.general](#)] paragraph 5 as follows:

~~If a *lambda-declarator* does not include a *parameter-declaration-clause*, it is as if () were inserted at the start of the *lambda-declarator*.~~ **A *lambda-expression's parameter-declaration-clause* is the *parameter-declaration-clause* of the *lambda-expression's lambda-declarator*, if any, or empty otherwise.** If the *lambda-declarator* does not include a *trailing-return-type*, the lambda return type is auto, which is deduced from return statements as described in 9.2.9.6 [[dcl.spec.auto](#)].

2485. Bit-fields in integral promotions

Section: 7.3.7 [[conv.prom](#)] **Status:** ready **Submitter:** Richard Smith **Date:** 2021-04-01

According to 7.3.7 [[conv.prom](#)] paragraph 5,

A prvalue for an integral bit-field (11.4.10 [[class.bit](#)]) can be converted to a prvalue of type `int` if `int` can represent all the values of the bit-field; otherwise, it can be converted to `unsigned int` if `unsigned int` can represent all the values of the bit-field. If the bit-field is larger yet, no integral promotion applies to it. If the bit-field has an enumerated type, it is treated as any other value of that type for promotion purposes.

This description has several problems. First, the “bit-field” semantic property only makes sense for glvalue expressions, so it's unclear why these rules are described as applying to a prvalue. Perhaps this should be rephrased as something like “An expression that was a bit-field glvalue prior to the application of the lvalue-to-rvalue conversion”?

Second, suppose that `char32_t` is wider than `int`. Per paragraph 2, a `char32_t` prvalue promotes to `unsigned long` (because `unsigned long` is necessarily at least 32 bits wide). But per paragraph 5, a `char32_t : 32` bitfield does not promote. This seems inconsistent.

Finally, it is not clear that the usual integral promotions are not applied to bit-fields. This should be made explicit.

Proposed resolution (approved by CWG 2023-02-07):

1. Insert a paragraph before 7.3.7 [[conv.prom](#)] paragraph 1 as follows:

For the purposes of 7.3.7 [[conv.prom](#)], a *converted bit-field* is a prvalue that is the result of an lvalue-to-rvalue conversion (7.3.2 [[conv.lval](#)]) applied to a bit-field (11.4.10 [[class.bit](#)]).

2. Change in 7.3.7 [[conv.prom](#)] paragraph 1 as follows:

A prvalue ~~of~~ **that is not a converted bit-field and has** an integer type other than `bool`, `char8_t`, `char16_t`, `char32_t`, or `wchar_t` whose integer conversion rank (6.8.6 [[conv.rank](#)]) is less than the rank of `int` can be converted to a prvalue of type `int` if `int` can represent all the values of the source type; otherwise, the source prvalue can be converted to a prvalue of type `unsigned int`.

3. Change in 7.3.7 [[conv.prom](#)] paragraph 4 as follows:

A prvalue of an unscoped enumeration type whose underlying type is fixed (9.7.1 [[dcl.enum](#)]) can be converted to a prvalue of its underlying type. Moreover, if integral promotion can be applied to its underlying type, a prvalue of an unscoped enumeration type whose underlying type is fixed can also be converted to a prvalue of the promoted underlying type. **[Note: A converted bit-field of enumeration type is treated as any other value of that type for promotion purposes. -- end note]**

4. Change in 7.3.7 [[conv.prom](#)] paragraph 5 as follows:

~~A prvalue for an integral bit-field (11.4.10 [[class.bit](#)])~~ **converted bit-field of integral type** can be converted to a prvalue of type `int` if `int` can represent all the values of the bit-field; otherwise, it can be converted to `unsigned int` if `unsigned int` can represent all the values of the bit-field. ~~If the bit-field is larger yet, no integral promotion applies to it. If the bit-field has enumeration type, it is treated as any other value of that type for promotion purposes.~~

5. Move 7.3.7 [conv.prom] paragraph 2 after paragraph 5 and change as follows:

A prvalue of type `char8_t`, `char16_t`, `char32_t`, or `wchar_t` (6.8.2 [basic.fundamental]) (including a converted bit-field that was not already promoted to `int` or unsigned `int` according to the rules above) can be converted to a prvalue of the first of the following types that can represent all the values of its underlying type: `int`, unsigned `int`, long `int`, unsigned long `int`, long long `int`, or unsigned long long `int`. If none of the types in that list can represent all the values of its underlying type, a prvalue of type `char8_t`, `char16_t`, `char32_t`, or `wchar_t` can be converted to a prvalue of, or its underlying type.

2519. Object representation of a bit-field

Section: 6.8.1 [basic.types.general] Status: ready Submitter: Jiang An Date: 2022-01-20

6.7.2 [intro.object] clearly implies that bit-fields are objects; paragraphs 8-9 contain phrases like “unless an object is a bit-field...” and “a non-bit-field subobject”. However, the definition of “object representation” in 6.8.1 [basic.types.general] paragraph 4 is,

The *object representation* of an object of type T is the sequence of N unsigned `char` objects taken up by the object of type T , where N equals `sizeof(T)`.

and thus fails to address bit-fields, which are not necessarily composed of a sequence of complete bytes.

The C Standard (6.2.6.1 paragraph 4) says,

Values stored in bit-fields consist of m bits, where m is the size specified for the bit-field. The object representation is the set of m bits the bit-field comprises in the addressable storage unit holding it.

Presumably similar wording could be adopted for C++.

Proposed resolution (approved by CWG 2023-01-06) [SUPERSEDED]:

Change in 6.8.1 [basic.types.general] paragraph 4 as follows:

The *object representation* of an object of a type T is the sequence of N unsigned `char` objects taken up by the a non-bit-field complete object of type T , where N equals `sizeof(T)`. The *value representation* of an object of a type T is the set of bits in the object representation of T that participate in representing a value of type T . The object and value representation of a non-bit-field complete object of type T are the bytes and bits, respectively, of the object corresponding to the object and value representation of its type. The object representation of a bit-field object is the sequence of N bits taken up by the object, where N is the width of the bit-field (11.4.10 [class.bit]). The value representation of a bit-field object is the set of bits in the object representation that participate in representing its value. Bits in the object representation of a type or object that are not part of the value representation are padding bits. For trivially copyable types, the value representation is a set of bits in the object representation that determines a value, which is one discrete element of an implementation-defined set of values. [Footnote: ...]

CWG 2023-02-06

Additional drafting is needed to constrain the definition to complete object types.

Proposed resolution (approved for C++26 by CWG 2023-02-06):

Change in 6.8.1 [[basic.types.general](#)] paragraph 4 as follows:

The *object representation* of ~~an object of~~ **a complete object** type τ is the sequence of N unsigned char objects taken up by ~~the~~ **a non-bit-field complete** object of type τ , where N equals `sizeof( $\tau$ )`. The *value representation* of ~~an object of~~ **a** type τ is the set of bits **in the object representation of τ** that participate in representing a value of type τ . **The object and value representation of a non-bit-field complete object of type τ are the bytes and bits, respectively, of the object corresponding to the object and value representation of its type. The object representation of a bit-field object is the sequence of N bits taken up by the object, where N is the width of the bit-field (11.4.10 [[class.bit](#)]). The value representation of a bit-field object is the set of bits in the object representation that participate in representing its value.** Bits in the object representation **of a type or object** that are not part of the value representation are padding bits. For trivially copyable types, the value representation is a set of bits in the object representation that determines a value, which is one discrete element of an implementation-defined set of values. [Footnote: ...]

2542. Is a closure type a structural type?

Section: 7.5.5.2 [[expr.prim.lambda.closure](#)] **Status:** ready **Submitter:** Zhihao Yuan **Date:** 2022-03-01

Consider:

```
template <auto V>
void foo() {}

void bar() {
    foo<[i = 3] { return i; }>();
}
```

It is unclear whether the data members of a closure type are public or private. This makes a difference, since it affects whether a closure type is a structural type or not (13.2 [[temp.param](#)] paragraph 7:

A structural type is one of the following:

- a scalar type, or
- an lvalue reference type, or
- a literal class type with the following properties:
 - all base classes and non-static data members are public and non-mutable and
 - the types of all bases classes and non-static data members are structural types or (possibly multi-dimensional) array thereof.

Proposed resolution (approved by CWG 2023-03-30):

Change in 7.5.5.2 [[expr.prim.lambda.closure](#)] paragraph 2 as follows:

... The closure type is not an aggregate type (9.4.2 [[decl.init.aggr](#)]) **and not a structural type (13.2 [[temp.param](#)])**. ...

2550. Type "reference to cv void" outside of a declarator

Section: 9.3.4.3 [[dcl.ref](#)] **Status:** ready **Submitter:** Jens Maurer **Date:** 2022-03-13

9.3.4.3 [[dcl.ref](#)] paragraph 1 specifies:

A declarator that specifies the type “reference to cv void” is ill-formed.

A *declarator* does not contain the leading *decl-specifier-seq* of a declaration, so the following example is not covered by the prohibition:

```
void f(void& x);
```

Proposed resolution (approved by CWG 2023-06-15):

Change in 9.3.4.3 [[dcl.ref](#)] paragraph 1 as follows:

~~A declarator that specifies~~ **Forming** the type “reference to cv void” is ill-formed.

2552. Constant evaluation of non-defining variable declarations

Section: 7.7 [[expr.const](#)] **Status:** ready **Submitter:** Hubert Tong **Date:** 2022-03-21

Paper [P2242](#) (Non-literal variables (and labels and gotos) in constexpr functions) added 7.7 [[expr.const](#)] bullet 5.2:

- a control flow that passes through a declaration of a variable with static (6.7.5.2 [[basic.stc.static](#)]) or thread (6.7.5.3 [[basic.stc.thread](#)]) storage duration;

It seems that block-scope extern (i.e. non-defining) declarations are covered by the above bullet, but only definitions should be in view here.

Proposed resolution (approved by CWG 2023-06-15):

1. Change in 7.7 [[expr.const](#)] bullet 5.2 as follows:

- a control flow that passes through a declaration of a **block** variable (**6.4.3** [[basic.scope.block](#)]) with static (6.7.5.2 [[basic.stc.static](#)]) or thread (6.7.5.3 [[basic.stc.thread](#)]) storage duration;

2663. Example for member redeclarations with *using-declarations*

Section: 9.9 [[namespace.udecl](#)] **Status:** ready **Submitter:** Shafik Yaghmour **Date:** 2022-11-28

Issue 36 was resolved by P1787R6, but no example was added.

Proposed resolution (approved by CWG 2023-06-13):

1. Add an example to 9.9 [[namespace.udecl](#)] paragraph 8 as follows:

```
[ Example:
struct C {
    int i;
};

struct D1 : C { };
struct D2 : C { };

struct D3 : D1, D2 {
    using D1::i;    // OK, equivalent to using C::i
    using D1::i;    // error: duplicate
    using D2::i;    // error: duplicate, also names C::i
};

-- end example ]
```

2. Change the example in 9.9 [[namespace.udecl](#)] paragraph 10 as follows:

```
using B::x;
using A::x;           // OK, hides struct B::x
using A::x;           // OK, does not conflict with previous using A::x
x = 99;               // assigns to A::x
struct x x1;           // x1 has class type B::x
}
```

CWG 2023-01-06

There is implementation divergence in handling this example.

CWG 2023-02-07

P1787R6 clarified that the example added to 9.9 [[namespace.udecl](#)] paragraph 10 is accepted, even in the non-function case.

2683. Default arguments for member functions of templated nested classes

Section: 9.3.4.7 [[dcl.fct.default](#)] **Status:** ready **Submitter:** Matthew House **Date:** 2023-01-11

Subclause 9.3.4.7 [[dcl.fct.default](#)] paragraph 6 specifies:

Except for member functions of class templates, the default arguments in a member function definition that appears outside of the class definition are added to the set of default arguments provided by the member function declaration in the class definition; the program is ill-formed if a default constructor (11.4.5.2 [[class.default.ctor](#)]), copy or move constructor (11.4.5.3 [[class.copy.ctor](#)]), or copy or move assignment operator (11.4.6 [[class.copy.assign](#)]) is so declared. Default arguments for a member function of a class template shall be specified on the initial declaration of the member function within the class template.

That rule appears to allow adding default arguments for member functions of classes that are nested within class templates, for example:

```
template<class> struct A { struct B { void c(int); }; };
template<class T> void A<T>::B::c(int = 0) {}
```

MSVC accepts; gcc and clang reject.

Proposed resolution (approved by CWG 2023-03-03):

Change in 9.3.4.7 [[dcl.fct.default](#)] paragraph 6 as follows:

Except for member functions of ~~class templates~~ **templated classes**, the default arguments in a member function definition that appears outside of the class definition are added to the set of default arguments provided by the member function declaration in the class definition; the program is ill-formed if a default constructor (11.4.5.2 [[class.default.ctor](#)]), copy or move constructor (11.4.5.3 [[class.copy.ctor](#)]), or copy or move assignment operator (11.4.6 [[class.copy.assign](#)]) is so declared. Default arguments for a member function of a **templated** class ~~template~~ shall be specified on the initial declaration of the member function within the **templated** class ~~template~~.

2697. Deduction guides using abbreviated function syntax

Section: 13.7.2.3 [[temp.deduct.guide](#)] **Status:** ready **Submitter:** CWG **Date:** 2023-02-11

It is unclear whether deduction guides can be expressed using abbreviated function syntax. Subclause 13.7.2.3 [[temp.deduct.guide](#)] paragraph 3 refers to the *restrictions* of a function's *parameter-declaration-clause*:

The same restrictions apply to the *parameter-declaration-clause* of a deduction guide as in a function declaration (9.3.4.6 [[dcl.fct](#)]). ...

However, that subclause is silent on the meaning of abbreviated function syntax when used for deduction guides. Furthermore, 9.3.4.6 [[dcl.fct](#)] paragraph 22 explicitly restricts the definition to function templates, which deduction guides are not:

An *abbreviated function template* is a function declaration that has one or more generic parameter type placeholders (9.2.9.6 [[dcl.spec.auto](#)]). ...

Arguably, the lack of template parameter names in abbreviated function syntax makes it less suitable to specify deduction guides.

CWG 2023-02-11

CWG solicits input from EWG whether abbreviated function syntax is intended to be used for deduction guides. See [cplusplus/papers#1465](#).

EWG 2023-05-11

CWG should clarify that abbreviated function syntax should not be permitted in deduction guides.

Proposed resolution (approved by CWG 2023-05-12):

Change in 13.7.2.3 [[temp.deduct.guide](#)] paragraph 3 as follows:

The same restrictions apply to the *parameter-declaration-clause* of a deduction guide as in a function declaration (9.3.4.6 [[dcl.fct](#)]), **except that a generic parameter type placeholder (9.2.9.6 [[dcl.spec.auto](#)]) shall not appear in the *parameter-declaration-clause* of a deduction guide.** The *simple-template-id* shall name a class template specialization. The *template-name* shall be the same identifier as the *template-name* of the *simple-template-id*. A *deduction-guide* shall inhabit the scope

to which the corresponding class template belongs and, for a member class template, have the same access. Two deduction guide declarations for the same class template shall not have equivalent *parameter-declaration-clauses* if either is reachable from the other.

2698. Using extended integer types with z suffix

Section: 5.13.2 [[lex.icon](#)] **Status:** ready **Submitter:** Mike Miller **Date:** 2023-02-17

Subclause 5.13.2 [[lex.icon](#)] paragraph 4 specifies:

If an *integer-literal* cannot be represented by any type in its list and an extended integer type (6.8.2 [[basic.fundamental](#)]) can represent its value, it may have that extended integer type. If all of the types in the list for the *integer-literal* are signed, the extended integer type shall be signed. If all of the types in the list for the *integer-literal* are unsigned, the extended integer type shall be unsigned. If the list contains both signed and unsigned types, the extended integer type may be signed or unsigned. A program is ill-formed if one of its translation units contains an *integer-literal* that cannot be represented by any of the allowed types.

This implies that an *integer-literal* with a z suffix can be of extended integer type, if the literal is larger than what is representable in `std::size_t`.

According to the author of the paper [P0330R8](#) (Literal Suffix for (signed) `size_t`) introducing the feature, this is unintentional; z should only yield `std::size_t` and its corresponding signed integer type.

See also the corresponding WG14 paper [N2998 Literal Suffixes for size_t](#).

Proposed resolution (reviewed by CWG 2023-03-03, approved by CWG 2023-05-12):

Change in 5.13.2 [[lex.icon](#)] paragraph 4 as follows:

If Except for integer-literals containing a size-suffix, if the value of an *integer-literal* cannot be represented by any type in its list and an extended integer type (6.8.2 [[basic.fundamental](#)]) can represent its value, it may have that extended integer type. If all of the types in the list for the *integer-literal* are signed, the extended integer type ~~shall be~~ **is** signed. If all of the types in the list for the *integer-literal* are unsigned, the extended integer type ~~shall be~~ **is** unsigned. If the list contains both signed and unsigned types, the extended integer type may be signed or unsigned. ~~A program is ill-formed if one of its translation units contains~~ **If an integer-literal that** cannot be represented by any of the allowed types, **the program is ill-formed. [Note: An integer-literal with a z or Z suffix is ill-formed if it cannot be represented by std::size_t. -- end note]**

Additional notes (February, 2023)

Alerted the chair of SG22 (C/C++ Liaison).

Forwarded to EWG at the request of the EWG chair via [cplusplus/papers#1467](#).

EWG 2023-05-11

The "z" suffixes mean `std::size_t` (or its corresponding signed type) only. The proposed resolution is accepted by EWG.

2699. Inconsistency of *throw-expression* specification

Section: 7.6.18 [[expr.throw](#)] **Status:** ready **Submitter:** Krystian Stasiowski **Date:** 2020-04-06

Subclause 7.6.18 [[expr.throw](#)] paragraph 2 and 3 specify:

Evaluating a *throw-expression* with an operand throws an exception (14.2 [[except.throw](#)]); the type of the exception object is determined by removing any top-level cv-qualifier s from the static type of the operand and adjusting the type from “array of T” or function type T to “pointer to T”.

A *throw-expression* with no operand rethrows the currently handled exception (14.4 [[except.handle](#)]). The exception is reactivated with the existing exception object; no new exception object is created. The exception is no longer considered to be caught.

This means that throwing a value of type `const char[3]` would throw a `char*` rather than `const char*`, which is not intended.

Proposed resolution (approved by CWG 2023-03-03):

Change in 7.6.18 [[expr.throw](#)] paragraph 2 through 4 as follows:

~~Evaluating a~~ **A** *throw-expression* with an operand throws an exception (14.2 [[except.throw](#)]); ~~the.~~
The array-to-pointer (7.3.3 [[conv.array](#)]) and function-to-pointer (7.3.4 [[conv.func](#)]) standard conversions are performed on the operand. The type of the exception object is determined by removing any top-level cv-qualifiers from the ~~static~~ type of the **(possibly converted)** operand ~~and adjusting the type from “array of T” or function type T to “pointer to T”.~~

A *throw-expression* with no operand rethrows the currently handled exception (14.4 [[except.handle](#)]). **If no exception is presently being handled, the function `std::terminate` is invoked (14.6.2 [[except.terminate](#)]).** ~~The~~ **Otherwise, the** exception is reactivated with the existing exception object; no new exception object is created. The exception is no longer considered to be caught.

~~If no exception is presently being handled, evaluating a *throw-expression* with no operand calls `std::terminate()` (14.6.2 [[except.terminate](#)]).~~

2708. Parenthesized initialization of arrays

Section: 9.4.1 [[decl.init.general](#)] **Status:** ready **Submitter:** Mike Miller **Date:** 2023-03-14

Consider:

```
const int arr[2](1,2);
```

This is accepted by all major implementations, yet 9.4.1 [[decl.init.general](#)] paragraph 13 prohibits it:

If the entity being initialized does not have class type, the *expression-list* in a parenthesized initializer shall be a single expression.

Presumably, this was an oversight when adding parenthesized aggregate initialization.

Proposed resolution (approved by CWG 2023-04-28):

Change in 9.4.1 [[dcl.init.general](#)] paragraph 13 as follows:

If the entity being initialized does not have class **or array** type, the *expression-list* in a parenthesized initializer shall be a single expression.

2710. Loops in constant expressions

Section: 7.7 [[expr.const](#)] **Status:** ready **Submitter:** Daniel Krügler **Date:** 2023-03-23

Iteration statements such as `while` and `for` loops are specified by equivalent code involving `goto` (8.6.2 [[stmt.while](#)] paragraph 2, 8.6.4 [[stmt.for](#)] paragraph 1, 8.6.5 [[stmt.ranged](#)] paragraph 1). The `goto` statement cannot be evaluated in constant expressions (7.7 [[expr.const](#)] bullet 5.30), thus `while` and `for` loops cannot be evaluated in constant expressions. Similar concerns arise for `continue` (8.7.3 [[stmt.cont](#)] paragraph 1).

However, that is neither intended nor existing practice.

Suggested resolution [SUPERSEDED]:

Change in 7.7 [[expr.const](#)] bullet 5.30 as follows:

An expression *E* is a *core constant expression* unless the evaluation of *E*, following the rules of the abstract machine (6.9.1 [[intro.execution](#)]), would evaluate one of the following:

- ...
- a `goto` statement (8.7.6 [[stmt.goto](#)]) **unless it is introduced by replacing an iteration-statement (8.6 [[stmt.iter](#)]) or a `continue` statement (8.7.3 [[stmt.cont](#)]) with its equivalent.**

CWG 2023-03-30

Keep the rule non-normative and non-exhaustive.

Proposed resolution (approved by CWG 2023-04-28):

Change in 7.7 [[expr.const](#)] bullet 5.30 as follows:

An expression *E* is a *core constant expression* unless the evaluation of *E*, following the rules of the abstract machine (6.9.1 [[intro.execution](#)]), would evaluate one of the following:

- ...
- a `goto` statement (8.7.6 [[stmt.goto](#)]). **[Note: A `goto` statement introduced by equivalence (Clause 8 [[stmt.stmt](#)]) is not in scope. For example, a `while` statement (8.6.2 [[stmt.while](#)]) can be executed during constant evaluation. -- end note]**

2711. Source for copy-initializing the exception object

Section: 7.6.18 [[expr.throw](#)] **Status:** ready **Submitter:** Brian Bi **Date:** 2023-03-26

Neither 14.2 [[except.throw](#)] nor 7.6.18 [[expr.throw](#)] specify the source for copy-initializing the exception object.

Proposed resolution (based on the resolution of issue 2699; approved by CWG 2023-06-12):

1. Change in 7.6.18 [[expr.throw](#)] paragraph 2 as follows:

Evaluating a *throw-expression* with an operand throws an exception (14.2 [[except.throw](#)]); the type of the exception object is determined by removing any top-level *cv-qualifiers* from the static type of the operand and adjusting the type from “array of T” or function type T to “pointer to T”. **The exception object is copy-initialized (9.4.1 [[dcl.init.general](#)]) from the (possibly converted) operand.**

2. Change in 14.2 [[except.throw](#)] paragraph 3 as follows:

Throwing an exception ~~copy~~-initializes ~~(9.4 [[dcl.init](#)], 11.4.5.3 [[class.copy.ctor](#)])~~ a temporary object, called the exception object. If the type of the exception object would be an incomplete type (**6.8.1 [[basic.types.general](#)]**), an abstract class type (11.7.4 [[class.abstract](#)]), or a pointer to an incomplete type other than cv void (**6.8.4 [[basic.compound](#)]**), the program is ill-formed.

2712. Simplify restrictions on built-in assignment operator candidates

Section: 12.2.2.3 [[over.match.oper](#)] **Status:** ready **Submitter:** Brian Bi **Date:** 2023-03-24

Subclause 12.2.2.3 [[over.match.oper](#)] paragraph 5 specifies:

For the built-in assignment operators, conversions of the left operand are restricted as follows:

- no temporaries are introduced to hold the left operand, and
- no user-defined conversions are applied to the left operand to achieve a type match with the left-most parameter of a built-in candidate.

The first bullet is redundant, because standard conversion sequences cannot bind “vq T&” (the type of the first parameter of a built-in assignment operator) to a temporary.

Proposed resolution (approved by CWG 2023-03-30):

Change in 12.2.2.3 [[over.match.oper](#)] paragraph 5 as follows:

For **the first parameter of** the built-in assignment operators, **only standard conversion sequences (12.2.4.2.2 [[over.ics.scs](#)]) are considered.** ~~conversions of the left operand are restricted as follows:~~

- ~~• no temporaries are introduced to hold the left operand, and~~
- ~~• no user-defined conversions are applied to the left operand to achieve a type match with the left-most parameter of a built-in candidate.~~

2713. Initialization of reference-to-aggregate from designated initializer list

Aggregates can be initialized by a designated initializer list, but references to aggregates cannot, although list-initialization of such with a regular *braced-init-list* is fine.

Subclause 9.4.5 [[dcl.init.list](#)] paragraph 3 specifies:

List-initialization of an object or reference of type T is defined as follows:

- If the *braced-init-list* contains a *designated-initializer-list*, T shall be an aggregate class. ...
- ...
- Otherwise, if T is a reference type, a prvalue is generated. The prvalue initializes its result object by copy-list-initialization from the initializer list. The prvalue is then used to direct-initialize the reference. The type of the prvalue is the type referenced by T, unless ...
- ...

Subclause 12.2.4.2.6 [[over.ics.list](#)] paragraph 2 specifies:

If the initializer list is a *designated-initializer-list*, a conversion is only possible if the parameter has an aggregate type that can be initialized from the initializer list according to the rules for aggregate initialization (9.4.2 [[dcl.init.aggr](#)]), in which case the implicit conversion sequence is a user-defined conversion sequence whose second standard conversion sequence is an identity conversion.

Proposed resolution (approved by CWG 2023-04-28):

1. Change in 9.4.5 [[dcl.init.list](#)] bullet 3.1 as follows:

- If the *braced-init-list* contains a *designated-initializer-list* **and T is not a reference type**, T shall be an aggregate class. ...
- ...

2. Change in 9.4.5 [[dcl.init.list](#)] bullet 3.9 as follows:

Otherwise, if the initializer list **is not a designated-initializer-list and** has a single element of type E and ...

3. Change in 9.4.5 [[dcl.init.list](#)] bullet 3.10 as follows:

[Example:

```
...  
const B& b2{a}; // error: cannot copy-list-initialize B temporary from A  
struct C { int x; };  
C&& c = { .x = 1 }; // OK
```

-- end example]

4. Change in 12.2.4.2.6 [[over.ics.list](#)] paragraph 2 as follows:

If the initializer list is a *designated-initializer-list* **and the parameter is not a reference**, a conversion is only possible if the parameter has an aggregate type that can be initialized from the initializer list according to the rules for aggregate initialization (9.4.2 [[dcl.init.aggr](#)]), in which case the implicit conversion sequence is a user-defined conversion sequence whose second standard conversion sequence is an identity conversion.

2715. "calling function" for parameter initialization may not exist

Section: 7.6.1.3 [[expr.call](#)] **Status:** ready **Submitter:** Brian Bi **Date:** 2023-04-02

Subclause 7.6.1.3 [[expr.call](#)] paragraph 6 specifies:

... The initialization and destruction of each parameter occurs within the context of the calling function. [Example 2: The access of the constructor, conversion functions or destructor is checked at the point of call in the calling function. If a constructor or destructor for a function parameter throws an exception, the search for a handler starts in the calling function; in particular, if the function called has a *function-try-block* (14.1 [[except.pre](#)]) with a handler that can handle the exception, this handler is not considered. —end example]

However, there is no calling function in the case where a function call appears in the initializer of a namespace-scope variable. Likewise, some constant expressions appearing in a *type-id* do not have calling functions, either. For example:

```
class C {
private:
    constexpr int C(int) {}
    friend void foo(int (*a)[1]) noexcept;
};

constexpr int bar(C) { return 1; }

void foo(int (&a)[bar(1)]) noexcept(bar(2) > 0); // presumably OK because of friendship
```

Proposed resolution (approved by CWG 2023-04-28):

Change in 7.6.1.3 [[expr.call](#)] paragraph 6 as follows:

... The initialization and destruction of each parameter occurs within the context of the **calling function** **full-expression (6.9.1 [[intro.execution](#)]) where the function call appears**. [Example 2: The access (**11.8.1 [[class.access.general](#)]**) of the constructor, conversion functions, or destructor is checked at the point of call **in the calling function**. If a constructor or destructor for a function parameter throws an exception, **the search for a handler starts in the calling function; in particular, if the function called has a** **any** *function-try-block* (14.1 [[except.pre](#)]) **of the called function** with a handler that can handle the exception, **this handler** is not considered. —end example]

2716. Rule about self-or-base conversion is normatively redundant

Section: 11.4.8.3 [[class.conv.fct](#)] **Status:** ready **Submitter:** Brian Bi **Date:** 2023-04-11

The rule in 11.4.8.3 [[class.conv.fct](#)] paragraph 4 is normatively redundant explanation:

A conversion function is never used to convert a (possibly cv-qualified) object to the (possibly cv-qualified) same object type (or a reference to it), to a (possibly cv-qualified) base class of that type (or a reference to it), or to cv void. [Footnote: These conversions are considered as standard conversions for the purposes of overload resolution (12.2.4.2 [[over.best.ics](#)], 12.2.4.2.5 [[over.ics.ref](#)]) and therefore initialization (9.4 [[dcl.init](#)]) and explicit casts (7.6.1.9 [[expr.static.cast](#)]). A conversion to void does not invoke any conversion function (7.6.1.9 [[expr.static.cast](#)]). Even

though never directly called to perform a conversion, such conversion functions can be declared and can potentially be reached through a call to a virtual conversion function in a base class. -- end footnote]

Proposed resolution (approved by CWG 2023-04-28):

Change in 11.4.8.3 [class.conv.fct] paragraph 4 as follows:

[**Note:** A conversion function is never ~~used to convert~~ **invoked for implicit or explicit conversions of an** a (possibly cv-qualified) object to the (possibly cv-qualified) same object type (or a reference to it), to a (possibly cv-qualified) base class of that type (or a reference to it), or to cv void. **Footnote:** These conversions are considered as standard conversions for the purposes of overload resolution (12.2.4.2 [over.best.ics], 12.2.4.2.5 [over.ics.ref]) and therefore initialization (9.4 [dcl.init]) and explicit casts (7.6.1.9 [expr.static.cast]). A conversion to void does not invoke any conversion function (7.6.1.9 [expr.static.cast]). Even though never directly called to perform a conversion, such conversion functions can be declared and can potentially be reached through a call to a virtual conversion function in a base class. ~~-- end footnote]~~ **-- end note]**

[Example: ...]

2717. Pack expansion for *alignment-specifier*

Section: 13.7.4 [temp.variadic] **Status:** ready **Submitter:** Jim X **Date:** 2023-04-11

Subclause 13.7.4 [temp.variadic] paragraph 11 specifies:

The instantiation of any other pack expansion produces a list of elements E1, E2, ... , EN.

Consider this example:

```
template<class ...T>
struct Align{
    alignas(T...) unsigned char buffer[128];
};
Align<int, short> a;
```

The pack expansion of the *alignment-specifier* yields alignas(int), alignas(short), an ill-formed list per the grammar in 9.12.1 [dcl.attr.grammar].

Proposed resolution (approved by CWG 2023-04-28):

Insert a new paragraph after 13.7.4 [temp.variadic] paragraph 9 as follows:

The instantiation of a sizeof... expression (7.6.2.5 [expr.sizeof]) produces an integral constant with value N.

The instantiation of an *alignment-specifier* with an ellipsis produces E₁ E₂ ... E_N.

2718. Type completeness for derived-to-base conversions

Section: 7.6.1.9 [[expr.static.cast](#)] **Status:** ready **Submitter:** Jim X **Date:** 2023-04-09

Issue 2310 clarified class completeness requirements for derived-to-base pointer conversions, but neglected the corresponding lvalue conversion.

Proposed resolution (approved by CWG 2023-04-28):

Change in 7.6.1.9 [[expr.static.cast](#)] paragraph 2 as follows:

An lvalue of type “cv1 B”, where B is a class type, can be cast to type “reference to cv2 D”, where D is a **complete** class derived (11.7 [[class.derived](#)]) from B, if cv2 is the same cv-qualification as, or greater cv-qualification than, cv1. ...

2719. Creating objects in misaligned storage

Section: 6.7.6 [[basic.align](#)] **Status:** ready **Submitter:** Jiang An **Date:** 2023-04-05

A non-allocating form of operator `new` can be used to create an object in storage that is not suitably aligned for the type of the object. Such attempts ought to be undefined behavior.

Proposed resolution (approved by CWG 2023-04-28):

1. Change in 6.7.6 [[basic.align](#)] paragraph 1 as follows:

Object types have alignment requirements (6.8.2 [[basic.fundamental](#)], 6.8.4 [[basic.compound](#)]) which place restrictions on the addresses at which an object of that type may be allocated. An alignment is an implementation-defined integer value representing the number of bytes between successive addresses at which a given object can be allocated. An object type imposes an alignment requirement on every object of that type; stricter alignment can be requested using the alignment specifier (9.12.2 [[dcl.align](#)]). **Attempting to create an object (6.7.2 [[intro.object](#)]) in storage that does not meet the alignment requirements of the object's type is undefined behavior.**

2. Change in 7.6.2.8 [[expr.new](#)] paragraph 22 as follows:

[*Note 11*: When the allocation function returns a value other than null, it must be a pointer to a block of storage in which space for the object has been reserved. The block of storage is assumed to be appropriately aligned (**6.7.6 [[basic.align](#)]**) and of the requested size. The address of the created object will not necessarily be the same as that of the block if the object is an array. —*end note*]

2720. Template validity rules for templated entities and alias templates

Section: 13.8.1 [[temp.res.general](#)] **Status:** ready **Submitter:** Richard Smith **Date:** 2023-03-29

Subclause 13.8.1 [[temp.res.general](#)] paragraph 6 specifies rules to determine when a template is valid, but the specification is in terms of “instantiation”. However, some kinds of templates (namely alias templates) are not instantiated.

Further, the rule discusses only templates, but should apply to any templated entity.

Proposed resolution (approved by CWG 2023-05-12):

Change in 13.8.1 [[temp.res.general](#)] paragraph 6 as follows:

The validity of a **template** **templated entity** may be checked prior to any instantiation. [Note 3: Knowing which names are type names allows the syntax of every template to be checked in this way. —end note]

The program is ill-formed, no diagnostic required, if:

- no valid specialization, ignoring *static_assert*-declarations that fail (9.1 [[dcl.pre](#)]), can be generated for a **template** **templated entity** or a substatement of a constexpr if statement (8.5.2 [[stmt.if](#)]) within a **template** **templated entity** and the **innermost enclosing** template is not instantiated, or
- **no specialization of an alias template (13.7.8 [[temp.alias](#)]) is valid and no specialization of the alias template is named in the program, or**
- any *constraint-expression* in the program, introduced or otherwise, has (in its normal form) an atomic constraint A where no satisfaction check of A could be well-formed and no satisfaction check of A is performed, or
- every valid specialization of a variadic template requires an empty template parameter pack, or
- a hypothetical instantiation of a **template** **templated entity** immediately following its definition would be ill-formed due to a construct (**other than a static_assert-declaration that fails**) that does not depend on a template parameter, or
- the interpretation of such a construct in the hypothetical instantiation is different from the interpretation of the corresponding construct in any actual instantiation of the **template** **templated entity**.

2721. When exactly is storage reused?

Section: 6.7.3 [[basic.life](#)] **Status:** ready **Submitter:** Richard Smith **Date:** 2023-03-23

Subclause 6.7.3 [[basic.life](#)] bullet 1.5 specifies:

The lifetime of an object o of type T ends when:

- if T is a non-class type, the object is destroyed, or
- if T is a class type, the destructor call starts, or
- the storage which the object occupies is released, or is reused by an object that is not nested within o (6.7.2 [[intro.object](#)]).

Consider the expression `new (p) T(x)`. Does the lifetime of *p end when p is returned from the allocation function, before x is evaluated? Or does the lifetime end when the constructor body starts executing, after x is evaluated?

The second option is conceivable for initialization by constructor and non-class types; for aggregate initialization, the first option must be used, because evaluation of x directly initializes a part of the resulting object. The first option is simpler to implement for constant evaluation.

Proposed resolution (approved by CWG 2023-05-12):

Change in 6.7.3 [[basic.life](#)] paragraph 1 as follows:

The lifetime of an object *o* of type *T* ends when:

- if *T* is a non-class type, the object is destroyed, or
- if *T* is a class type, the destructor call starts, or
- the storage which the object occupies is released, or is reused by an object that is not nested within *o* (6.7.2 [[intro.object](#)]).

When evaluating a *new-expression*, storage is considered reused after it is returned from the allocation function, but before the evaluation of the *new-initializer* (7.6.2.8 [[expr.new](#)]). [[intro.object](#)].

Example:

```
struct S {
    int m;
};

void f() {
    S x{1};
    new(&x) S(x.m);    // undefined behavior
}
```

-- end example]

2722. Temporary materialization conversion for noexcept operator

Section: 7.6.2.7 [[expr.unary.noexcept](#)] **Status:** ready **Submitter:** Brian Bi **Date:** 2023-04-24

It is unclear whether `noexcept(A())` applies the temporary materialization conversion to the prvalue `A()`. The resolution of issue 1354 suggests that it does so that the destructor is (notionally) invoked.

Proposed resolution (approved by CWG 2023-05-12):

Change in 7.6.2.7 [[expr.unary.noexcept](#)] paragraph 3 as follows:

If the operand is a prvalue, the temporary materialization conversion (7.3.5 [[conv.rval](#)]) is applied. The result of the `noexcept` operator is true unless the ~~expression~~ **full-expression of the operand** is potentially-throwing (14.5 [[except.spec](#)]).

2723. Range of representable values for floating-point types

Section: 6.8.2 [[basic.fundamental](#)] **Status:** ready **Submitter:** Jiang An **Date:** 2023-04-21

The range of representable values is defined for integer types, but not for floating-point types. This term is used in 5.13.4 [[lex.fcon](#)] paragraph 3 as well as in the library, e.g. in 22.13.3 [[charconv.from.chars](#)] and 30.4.3.2.3 [[facet.num.get.virtuals](#)].

The C standard contains a suitable definition that we should inherit.

Proposed resolution (approved by CWG 2023-05-12):

Add a new paragraph after 6.8.2 [[basic.fundamental](#)] paragraph 12 as follows:

... Except as specified in 6.8.3 [[basic.extended.fp](#)], the object and value representations and accuracy of operations of floating-point types are implementation-defined.

The minimum range of representable values for a floating-point type is the most negative finite floating-point number representable in that type through the most positive finite floating-point number representable in that type. In addition, if negative infinity is representable in a type, the range of that type is extended to all negative real numbers; likewise, if positive infinity is representable in a type, the range of that type is extended to all positive real numbers. [Note: Since negative and positive infinity are representable in ISO/IEC/IEEE 60559 formats, all real numbers lie within the range of representable values of a floating-point type adhering to ISO/IEC/IEEE 60559.]

2724. Clarify rounding for arithmetic right shift

Section: 7.6.7 [[expr.shift](#)] **Status:** ready **Submitter:** Eisenwave **Date:** 2023-04-07

(From [editorial issue 6225](#).)

Subclause 7.6.7 [[expr.shift](#)] paragraph 3 specifies:

The value of $E1 \gg E2$ is $E1/2^{E2}$, rounded down.

It is unclear whether "rounded down" means "towards zero" or "towards negative infinity".

Proposed resolution (approved by CWG 2023-05-12):

Change in 7.6.7 [[expr.shift](#)] paragraph 3 as follows:

The value of $E1 \gg E2$ is $E1/2^{E2}$, rounded ~~down~~ **towards negative infinity**.

2729. Meaning of *new-type-id*

Section: 7.6.2.8 [[expr.new](#)] **Status:** ready **Submitter:** Jim X **Date:** 2023-02-06

Subclause 7.6.2.8 [[expr.new](#)] paragraph 1 introduces the grammar non-terminal *new-type-id*, but never specifies its meaning.

Proposed resolution (approved by CWG 2023-06-12):

1. Change in 7.6.2.8 [[expr.new](#)] paragraph 1 as follows:

The *new-expression* attempts to create an object of the *type-id* ~~(9.3.2 [[decl.name](#)])~~ or *new-type-id* **(9.3.2 [[decl.name](#)])** to which it is applied. The type of that object is the allocated type. This

type shall be a complete object type (6.8.1 [basic.types.general]), but not an abstract class type (11.7.4 [class.abstract]) or array thereof (6.7.2 [intro.object]).

2. Change in 9.3.2 [dcl.name] paragraph 1 as follows:

To specify type conversions explicitly, and as an argument of sizeof, alignof, new, or typeid, the name of a type shall be specified. This can be done with a *type-id* or *new-type-id* (7.6.2.8 [expr.new]), which is syntactically a declaration for a variable or function of that type that omits the name of the entity.

2732. Can importable headers react to preprocessor state from point of import?

Section: 10.3 [module.import] **Status:** ready **Submitter:** Xu Chuanqi **Date:** 2023-05-25

Subclause 10.3 [module.import] paragraph 5 specifies:

A module-import-declaration that specifies a header-name H imports a synthesized header unit, which is a translation unit formed by applying phases 1 to 7 of translation (5.2 [lex.phases]) to the source file or header nominated by H, which shall not contain a module-declaration. [Note 2: All declarations within a header unit are implicitly exported (10.2 [module.interface]), and are attached to the global module (10.1 [module.unit]). —end note]

It is unclear whether the contents of header units can vary depending on the set of defined macros at the point where the import (or #include) appears.

Proposed resolution (approved by CWG 2023-06-13):

Change in 10.3 [module.import] paragraph 5 as follows:

[Note 2: **A header unit is a separate translation unit with an independent set of defined macros.** All declarations within a header unit are implicitly exported (10.2 [module.interface]), and are attached to the global module (10.1 [module.unit]). —end note]

2750. construct_at without constructor call

Section: 7.7 [expr.const] **Status:** ready **Submitter:** CWG **Date:** 2023-06-16

Subclause 7.7 [expr.const] paragraph 6 talks about invoking the "underlying constructor", but there is no constructor when creating an aggregate or a scalar type.

Proposed resolution:

For the purposes of determining whether an expression E is a core constant expression, the evaluation of the body of a member function of std::allocator<T> as defined in 20.2.10.2 [allocator.members], where T is a literal type, is ignored. Similarly, the evaluation of the body of std::construct_at or std::ranges::construct_at is considered to include only the ~~underlying constructor call~~ **initialization of the T object** if the first argument (of type T*) points to storage allocated with std::allocator<T> or to an object whose lifetime began within the evaluation of E.

